TECNOLÓGICO NACIONAL DE MÉXICO

Introducción a DevOps con Contenedores y Docker

Asignatura: Desarrollo de aplicaciones móviles híbridas

Integrantes:
Armand Leonardo Rodríguez Olivera
Alejandra Priscila Mendez Couoh

TECNOLÓGICO NACIONAL DE MÉXICO

Objetivo

Comprender los conceptos básicos de DevOps, contenedores y Docker, y realizar una primera práctica de creación y ejecución de un contenedor.

TECNOLÓGICO
NACIONAL DE MÉXICO

Documento de apoyo para la actividad académica

1. Investigación

1.1 ¿Qué es DevOps?

DevOps es una forma de trabajo que integra las actividades de **desarrollo de software (Development)** y **operaciones de TI (Operations)**. Su propósito es que un equipo pueda desarrollar, probar, entregar y mantener aplicaciones de manera más rápida, confiable y automatizada.

DevOps no es solamente una herramienta o un programa; es una combinación de cultura, prácticas y herramientas que busca mejorar la colaboración entre las personas involucradas en el desarrollo y operación de un sistema.

En palabras sencillas: DevOps busca que desarrollar, probar y poner en funcionamiento un software sea un proceso colaborativo, automatizado y continuo.

1.2 ¿Qué relación existe entre desarrollo y operaciones?

El **desarrollo** se enfoca principalmente en crear y modificar el software, mientras que **operaciones** se encarga de que ese software pueda ejecutarse, mantenerse disponible y funcionar correctamente en los ambientes correspondientes.

Desarrollo	Operaciones
Programa nuevas funciones	Administra los ambientes
Corrige errores	Supervisa los sistemas
Realiza pruebas	Realiza despliegues
Modifica el código	Mantiene los servicios funcionando
Prepara nuevas versiones	Vigila rendimiento y disponibilidad

En un modelo DevOps ambas áreas dejan de trabajar como departamentos aislados y colaboran durante todo el ciclo de vida de la aplicación. Esto permite reducir problemas durante la entrega y mejorar la velocidad de publicación.

Ejemplo: Código → Integración → Pruebas → Construcción → Despliegue → Monitoreo → Retroalimentación.

1.3 ¿Qué es un contenedor?

Un **contenedor** es un entorno aislado donde se ejecuta una aplicación junto con los archivos, bibliotecas y configuraciones que necesita para funcionar. La principal ventaja es que la aplicación puede ejecutarse de una manera más consistente independientemente del equipo donde se encuentre el contenedor.

Docker define un contenedor como una **instancia ejecutable de una imagen**. Los contenedores pueden crearse, iniciarse, detenerse, trasladarse o eliminarse.

1.4 ¿Qué es Docker?

Docker es una plataforma utilizada para desarrollar, empaquetar, distribuir y ejecutar aplicaciones mediante contenedores. Permite separar una aplicación de la infraestructura donde se ejecuta.

Docker proporciona herramientas para administrar imágenes, contenedores, redes, volúmenes y registros de imágenes. Docker Desktop facilita el uso de Docker en Windows, macOS y Linux.

1.5 ¿Qué es una imagen?

Una **imagen Docker** es una plantilla de solo lectura que contiene los elementos necesarios para crear un contenedor. Una imagen puede incluir un sistema base, bibliotecas, dependencias, archivos de configuración y código de una aplicación.

Las imágenes están formadas por capas, lo que permite reutilizar partes de una imagen y hacer más eficientes las construcciones.

1.6 ¿Qué es un contenedor Docker?

Un **contenedor Docker** es una instancia de una imagen que se encuentra ejecutándose o preparada para ejecutarse.

```
Imagen: python:3.13
          |
          +-- Contenedor 1
          +-- Contenedor 2
          +-- Contenedor 3
```

1.7 Diferencia entre imagen y contenedor

Imagen	Contenedor
Es una plantilla.	Es una instancia de la imagen.
Es de solo lectura.	Tiene una capa de escritura.
Sirve para crear contenedores.	Ejecuta la aplicación.
Puede almacenarse en un registro.	Puede iniciarse, detenerse o eliminarse.
No es una aplicación ejecutándose.	Es el proceso/entorno ejecutándose.

Analogía: Imagen = receta; contenedor = platillo preparado utilizando esa receta.

1.8 Ventajas de utilizar contenedores

- **Portabilidad:** una aplicación puede ejecutarse en diferentes ambientes con el soporte necesario.
- **Consistencia:** el mismo ambiente puede utilizarse durante desarrollo, pruebas y despliegue.
- **Rapidez:** los contenedores suelen ser más ligeros que una máquina virtual completa.
- **Aislamiento:** los procesos y recursos pueden mantenerse separados de otros contenedores.
- **Escalabilidad:** es posible ejecutar múltiples instancias de una aplicación.
- **Automatización:** Docker puede integrarse con procesos de integración y entrega continua.
- **Reproducibilidad:** se pueden crear ambientes a partir de imágenes y Dockerfiles.

• **Integración con DevOps:** facilita ambientes reproducibles y procesos automatizados.

2. Práctica con Docker

Paso 1. Instalar Docker

En Windows se puede utilizar Docker Desktop. Una vez instalado, se debe iniciar Docker Desktop.

Documentación oficial: https://docs.docker.com/desktop/

Paso 2. Verificar que Docker funcione

```
docker --version
docker info
```

El primer comando muestra la versión instalada y el segundo proporciona información del entorno Docker.

Paso 3. Ejecutar un contenedor de prueba

```
docker run hello-world
```

Si la imagen no existe localmente, Docker puede obtenerla de un registro y posteriormente crear y ejecutar el contenedor. El resultado debe incluir un mensaje similar a:

```
Hello from Docker!
```

Esto confirma que Docker pudo obtener la imagen, crear el contenedor, ejecutarlo y mostrar el resultado.

Paso 4. Identificar la imagen utilizada

```
docker images
```

Se podrá observar la imagen **hello-world**, su etiqueta y su identificador.

Paso 5. Observar el estado del contenedor

`docker ps docker ps -a` docker ps muestra los contenedores activos. docker ps -a muestra también los contenedores detenidos.

Paso 6. Detener y volver a ejecutar un contenedor

```
docker run -d --name mi-nginx -p 8080:80 nginx
docker ps docker stop mi-nginx docker ps docker
start mi-nginx docker ps
```

-d ejecuta en segundo plano; **--name** asigna un nombre; **-p 8080:80** relaciona el puerto 8080 del equipo con el puerto 80 del contenedor; **nginx** es la imagen utilizada.

Paso 7. ¿Qué función cumple un Dockerfile?

Un **Dockerfile** es un archivo de texto que contiene instrucciones para construir una imagen Docker. Puede definir la imagen base, directorio de trabajo, archivos que se copian, dependencias, variables, puertos y comando de inicio.

Paso 8. Elaborar un Dockerfile sencillo

Se propone un pequeño proyecto en Python relacionado con un entorno de desarrollo, que permite comprobar de forma sencilla la creación y ejecución de una imagen.

```
docker-practica/
|
+-- app.py
+-- Dockerfile
```

app.py

```
print("Aplicación ejecutándose dentro de Docker")
print("Práctica de DevOps con contenedores")
print("Contenedor funcionando correctamente")
```

Dockerfile

```
FROM python:3.13-slim

WORKDIR /app

COPY app.py .

CMD ["python", "app.py"]
```

Paso 9. Construir una imagen a partir del Dockerfile

```
cd ruta\docker-practica docker build -t
mi-aplicacion-docker .
```

La opción **-t** asigna el nombre a la imagen. El punto indica que se utiliza la carpeta actual como contexto de construcción.

Paso 10. Ejecutar un contenedor utilizando la imagen creada

```
docker images
docker run --name mi-contenedor mi-aplicacion-docker
```

El resultado esperado es:

```
Aplicación ejecutándose dentro de Docker
Práctica de DevOps con contenedores
Contenedor funcionando correctamente
```

Paso 11. Comprobar que el contenedor funcione correctamente

```
docker ps -a
```

El contenedor puede aparecer como **Exited (0)**. En este caso no significa que haya fallado: el programa imprime sus mensajes y termina. El código de salida 0 indica que el proceso terminó correctamente.

3. Flujo completo de la práctica

```
                    DEVOPS
|                     v
Desarrollo
|                     v
Código fuente
|                     v
Dockerfile
|                     v
docker build
|                     v
                   IMAGEN
                     |
v                  docker
run                      |
v                CONTENEDOR
|                     v
Aplicación
|                     v
Verificación
```

4. Relación entre DevOps y Docker

Docker puede utilizarse como parte de una estrategia DevOps porque facilita la creación de ambientes reproducibles y la automatización de diferentes etapas del ciclo de desarrollo.

```
Desarrollador       |
v    Código       |
v  Git / GitHub       |
v  Dockerfile       |
v Construcción de
imagen       |       v
Pruebas       |       v
Contenedor       |
v  Despliegue       |
v  Monitoreo
```

5. Conclusión

DevOps representa una forma de trabajo que busca integrar el desarrollo y las operaciones para mejorar la velocidad, calidad, colaboración y confiabilidad en la entrega de software. Su objetivo no es simplemente utilizar determinadas herramientas, sino establecer una cultura de colaboración y automatización durante todo el ciclo de vida de una aplicación.

Los contenedores son importantes dentro de este enfoque porque permiten empaquetar una aplicación junto con los elementos que necesita para ejecutarse. Docker proporciona las herramientas necesarias para crear imágenes, ejecutar contenedores y administrar estos ambientes.

Durante la práctica se comprobó la diferencia entre una imagen y un contenedor: la imagen funciona como una plantilla, mientras que el contenedor es una instancia ejecutable de dicha imagen. También se creó un Dockerfile, se construyó una imagen propia y posteriormente se ejecutó un contenedor utilizando esa imagen.

Por lo tanto, Docker puede ayudar a que un equipo de desarrollo trabaje con ambientes más consistentes y reproducibles, facilitando posteriormente la integración con procesos de automatización y CI/CD.

6. Comandos principales utilizados

```
docker --version
docker info

docker run hello-world

docker images
docker ps
docker ps -a

docker run -d --name mi-nginx -p 8080:80 nginx
docker stop mi-nginx docker start mi-nginx

docker build -t mi-aplicacion-docker . docker run
--name mi-contenedor mi-aplicacion-docker
```

TECNOLÓGICO NACIONAL DE MÉXICO

7. Bibliografía

Docker Docs. *What is Docker?*. https://docs.docker.com/get-started/docker-overview/

Docker Docs. *What is an image?*. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/

Docker Docs. *Dockerfile reference*. https://docs.docker.com/reference/dockerfile/

Docker Docs. *Writing a Dockerfile*. https://docs.docker.com/get-started/docker-concepts/building-images/writing-a-dockerfile/

Docker Docs. *Build, tag, and publish an image*. https://docs.docker.com/get-started/docker-concepts/building-images/build-tag-and-publish-an-image/

Docker Docs. *Docker Desktop*. https://docs.docker.com/desktop/

Amazon Web Services. *¿Qué es DevOps?*. https://aws.amazon.com/es/devops/what-is-devops/

TECNOLÓGICO NACIONAL DE MÉXICO